Nome: Lucas Lima de Barros Lica. **Turma:** 2°AMS.

Laboratório 03 — Padrão de Projeto Observer

Atividade 1 — Analisando o problema

4.1 — Qual é a responsabilidade da classe Pedido?

R: Na solução inicial, a classe Pedido é responsável por controlar o próprio estado, principalmente o atributo status. Além disso, ela também executa diretamente as ações de notificação quando o status muda: envia um e-mail, envia um SMS e registra a alteração em um log. Portanto, Pedido concentra responsabilidades que poderiam ser separadas. Esse excesso de responsabilidades cria forte acoplamento entre a classe e os mecanismos de notificação.

4.2 — Quais ações são executadas dentro de mudarStatus()?

R: O método mudarStatus() primeiro atualiza o atributo status. Em seguida, envia uma mensagem de e-mail, envia uma mensagem de SMS e registra a mudança em um log interno. Todas essas ações são executadas diretamente dentro do próprio método.

4.3 — O que aconteceria se a empresa pedisse uma notificação via WhatsApp?

R: Seria necessário alterar o método mudarStatus() para acrescentar uma nova ação responsável pelo WhatsApp. Isso faria Pedido conhecer mais uma forma concreta de notificação e aumentaria ainda mais o acoplamento. Quanto mais canais forem adicionados, maior ficará a quantidade de responsabilidades concentradas em Pedido.

4.4 — Seria necessário modificar a classe Pedido?

R: Sim. Na solução inicial, Pedido precisaria ser modificado para cada novo canal de comunicação ou nova regra de notificação. Essa necessidade é justamente um dos problemas que o padrão Observer procura resolver, pois uma solução baseada em observadores permite adicionar novas reações sem modificar o Subject.

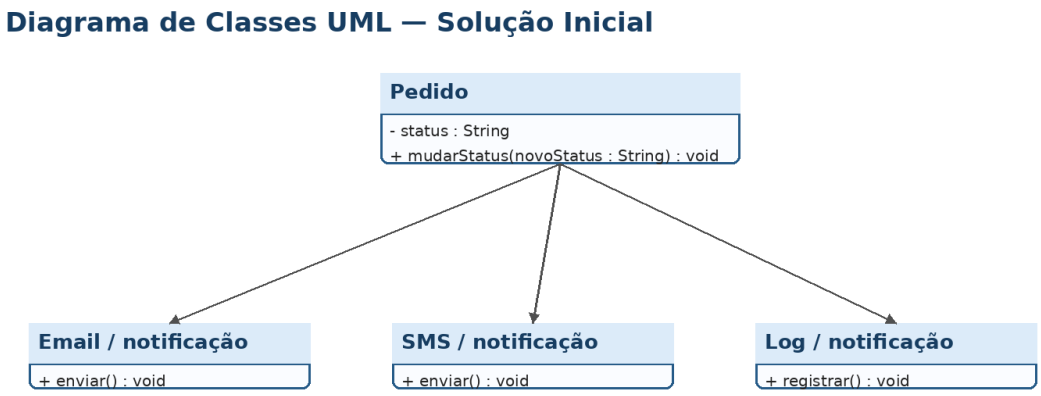
4.5 — O que aconteceria se o cliente quisesse desativar as notificações por SMS?

R: Seria necessário alterar Pedido ou acrescentar uma condição específica para impedir o envio do SMS. Isso tornaria o método ainda mais complexo e manteria a classe responsável por regras que pertencem ao mecanismo de notificação. Com Observer, o SMSObserver poderia simplesmente ser removido da lista de observadores.

Atividade 2 — Modelagem inicial

5 — Construa um Diagrama de Classes UML para a solução inicial.

R: A solução inicial possui Pedido como a classe central. Ela contém o atributo status e o método mudarStatus(). O método também executa diretamente as ações de e-mail, SMS e log. O diagrama abaixo representa conceitualmente esse acoplamento.



Atividade 3 — Criando a interface Observer

7 — Qual é a função da interface Observer?

R: A interface Observer define o contrato comum para todos os objetos que precisam ser avisados quando o estado do Subject muda. No laboratório, esse contrato é representado pelo método atualizar(status). Cada observador concreto implementa esse método e define sua própria reação.

```
public interface Observer {
    void atualizar(String status);
}
```

Atividade 4 — Criando a interface Subject

8 — Qual é a função da interface Subject?

R: A interface Subject define o contrato do objeto observado. Ela permite adicionar observadores, remover observadores e notificar todos os observadores cadastrados. Dessa maneira, o Subject trabalha com a abstração Observer, sem depender diretamente das classes concretas.

```
public interface Subject {
    void adicionarObserver(Observer observer);
    void removerObserver(Observer observer);
    void notificarObservers();
}
```

Atividade 5 — Criando o Subject concreto

9 — Como Pedido deve ser implementado utilizando Subject?

R: Pedido deve implementar Subject e manter uma lista de objetos do tipo Observer. Quando mudarStatus() altera o estado, o método chama notificarObservers(). Esse método percorre a lista e chama atualizar(status) em

cada observador. Assim, Pedido não conhece diretamente EmailObserver, SMSObserver, LogObserver ou WhatsAppObserver.

```
import java.util.ArrayList;
import java.util.List;

public class Pedido implements Subject {
    private List<Observer> observers = new ArrayList<>();
    private String status;

    @Override
    public void adicionarObserver(Observer observer) {
        observers.add(observer);
    }

    @Override
    public void removerObserver(Observer observer) {
        observers.remove(observer);
    }

    @Override
    public void notificarObservers() {
        for (Observer observer : observers) {
            observer.atualizar(status);
        }
    }

    public void mudarStatus(String novoStatus) {
        this.status = novoStatus;
        notificarObservers();
    }
}
```

Atividade 6 — Criando os observadores concretos

10 — Quais são as responsabilidades dos observadores concretos?

R: Cada observador concreto recebe a alteração de status e executa sua própria ação. EmailObserver envia e-mail, SMSObserver envia SMS e LogObserver registra a alteração. Posteriormente, WhatsAppObserver poderá enviar uma mensagem pelo WhatsApp. Essa separação permite que cada classe tenha uma responsabilidade específica.

```
public class EmailObserver implements Observer {
    @Override
    public void atualizar(String status) {
        System.out.println(
            "Enviando e-mail: status alterado para " + status
        );
    }
}

public class SMSObserver implements Observer {
    @Override
    public void atualizar(String status) {
```

```
        System.out.println(
            "Enviando SMS: status alterado para " + status
        );
    }
}

public class LogObserver implements Observer {
    @Override
    public void atualizar(String status) {
        System.out.println(
            "LOG: status alterado para " + status
        );
    }
}
```

Atividade 7 — Testando o sistema

11 — Qual é o resultado esperado e o que ele demonstra?

R: O programa deve notificar os três observadores cadastrados sempre que o status mudar. Isso demonstra que um único Subject pode possuir vários observadores e que cada um recebe a mesma atualização para executar sua própria ação.

```
public class Main {
    public static void main(String[] args) {
        Pedido pedido = new Pedido();

        pedido.adicionarObserver(new EmailObserver());
        pedido.adicionarObserver(new SMSObserver());
        pedido.adicionarObserver(new LogObserver());

        pedido.mudarStatus("Pedido enviado");
        System.out.println();
        pedido.mudarStatus("Pedido entregue");
    }
}
```

R: Saída esperada: primeiro são exibidas as mensagens de e-mail, SMS e log referentes a "Pedido enviado". Depois, as três notificações são repetidas para "Pedido entregue". O importante é que Pedido apenas dispara a atualização, enquanto cada Observer executa sua ação.

Atividade 8 — Removendo um observador

12 — O que a possibilidade de adicionar e remover observadores em tempo de execução representa em termos de flexibilidade?

R: Representa flexibilidade e baixo acoplamento. Os observadores podem ser ativados ou removidos dinamicamente sem modificar a classe Pedido. Por exemplo, se o SMSObserver for removido antes da segunda mudança de status, somente os observadores que permanecerem na lista serão notificados.

```
SMSObserver smsObserver = new SMSObserver();
```

```
pedido.adicionarObserver(new EmailObserver());
pedido.adicionarObserver(smsObserver);
pedido.adicionarObserver(new LogObserver());

pedido.mudarStatus("Pedido enviado");

pedido.removerObserver(smsObserver);

pedido.mudarStatus("Pedido entregue");
```

Atividade 9 — Novo tipo de observador

13 — Como adicionar WhatsAppObserver sem modificar as classes existentes?

R: Deve ser criada uma nova classe chamada WhatsAppObserver que implemente Observer. Ela deve sobrescrever atualizar(status) e realizar a ação correspondente ao WhatsApp. Depois, basta adicioná-la ao Pedido em Main. Observer, Subject, Pedido, EmailObserver, SMSObserver e LogObserver permanecem inalterados.

```
public class WhatsAppObserver implements Observer {
    @Override
    public void atualizar(String status) {
        System.out.println(
            "Enviando WhatsApp: status alterado para " + status
        );
    }
}
```

Atividade 10 — Entendendo o fluxo

14.1 — Quem decide quando notificar?

R: Quem decide é o Subject concreto, Pedido. Quando mudarStatus() altera o estado, ele chama notificarObservers(), iniciando a comunicação com todos os observadores cadastrados.

14.2 — Quem decide quais ações executar?

R: Cada Observer concreto decide sua própria ação. EmailObserver envia e-mail, SMSObserver envia SMS, LogObserver registra o log e WhatsAppObserver envia a mensagem pelo WhatsApp.

14.3 — Pedido conhece as classes concretas dos observadores?

R: Não. Pedido conhece somente a abstração Observer e mantém uma lista de objetos desse tipo. Por isso, novas implementações podem ser adicionadas sem modificar Pedido.

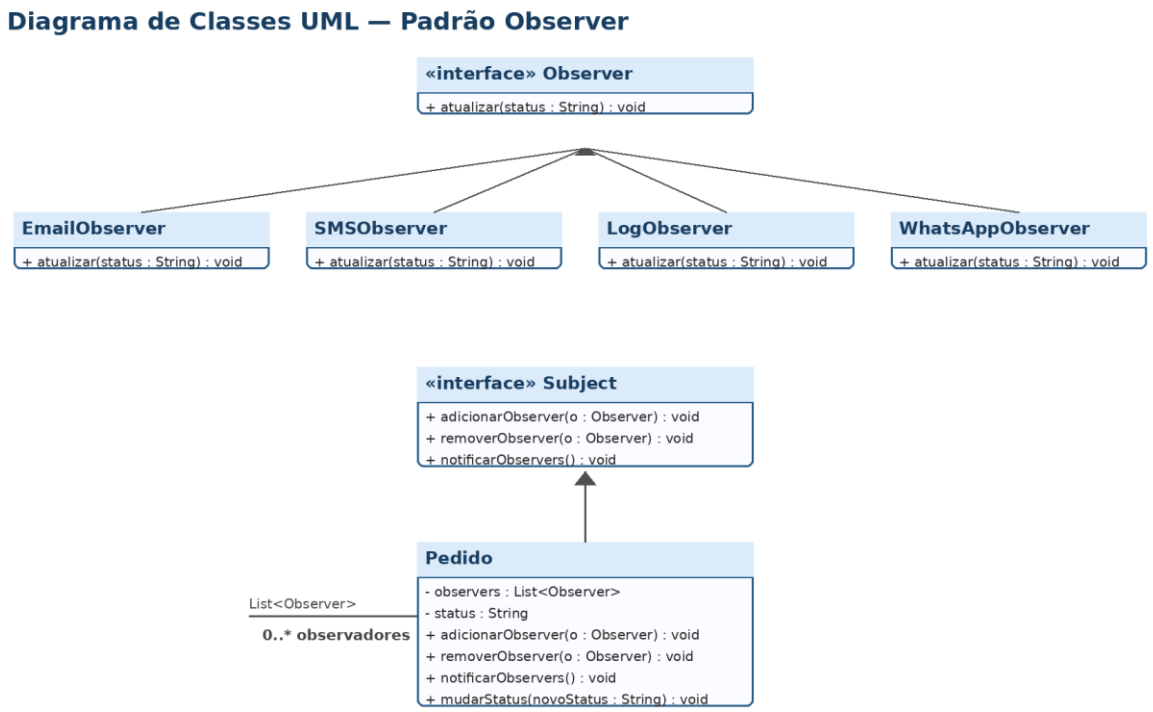
14.4 — O que aconteceria se dois observadores fizessem a mesma ação?

R: Os dois seriam notificados normalmente e poderiam executar a mesma ação. O padrão Observer não exige que cada observador tenha uma reação diferente; ele apenas garante que todos os observadores cadastrados recebam a atualização.

Atividade 11 — Diagrama de Classes final

15 — Como fica o Diagrama de Classes completo?

R: O modelo final possui duas interfaces principais. Observer define atualizar(status), enquanto Subject define as operações de adicionar, remover e notificar observadores. EmailObserver, SMSObserver, LogObserver e WhatsAppObserver implementam Observer. Pedido implementa Subject e mantém uma associação de 0..* observadores por meio de List<Observer>. Essa estrutura representa corretamente a relação um-para-muitos do padrão Observer.



Atividade 12 — Desafio: múltiplos pedidos

16 — Como implementar múltiplos pedidos com conjuntos diferentes de observadores?

R: É possível criar vários objetos Pedido, porque cada instância possui sua própria lista de observadores. Assim, um pedido pode ter e-mail, SMS e log, enquanto outro pode possuir apenas e-mail e log. A alteração de um pedido não notifica os observadores de outro pedido.

```
Pedido pedido1 = new Pedido();
pedido1.adicionarObserver(new EmailObserver());
pedido1.adicionarObserver(new SMSObserver());
pedido1.adicionarObserver(new LogObserver());

Pedido pedido2 = new Pedido();
pedido2.adicionarObserver(new EmailObserver());
pedido2.adicionarObserver(new LogObserver());
```

```
pedido1.mudarStatus("Pedido enviado");
pedido2.mudarStatus("Pedido entregue");
```

R: No exemplo, pedido1 notifica e-mail, SMS e log, enquanto pedido2 notifica somente e-mail e log. Nenhuma classe existente precisa ser modificada.

Atividade 13 — Observer × Strategy × Factory Method

17 — Qual representa comportamento, qual representa criação e qual representa notificação?
R: Strategy é utilizado para encapsular alternativas de comportamento ou algoritmos. No laboratório de descontos, cada estratégia representa uma forma diferente de calcular ou aplicar uma regra. Factory Method é utilizado para organizar a criação de objetos e permitir que subclasses determinem qual produto concreto será criado. Observer, por sua vez, organiza a comunicação entre objetos quando ocorre uma mudança de estado. Em termos simples: Strategy responde "qual comportamento utilizar?"; Factory Method responde "qual objeto criar?"; Observer responde "quem deve ser avisado quando algo mudar?".

17.1 — Como funciona a cardinalidade no Observer em comparação com Strategy?
R: No Strategy, o contexto normalmente utiliza uma estratégia selecionada para executar determinado comportamento, caracterizando uma relação de escolha de estratégia. No Observer, um único Subject pode manter vários observadores simultaneamente. Portanto, Observer apresenta claramente uma relação um-para-muitos: um Pedido pode notificar e-mail, SMS, log, WhatsApp e outros observadores.

Atividade 14 — Reflexão

Questão 1 — O que é o padrão Observer?
R: Observer é um padrão de projeto comportamental que define uma dependência um-para-muitos entre objetos. Quando o objeto observado muda de estado, seus observadores são notificados automaticamente para que possam reagir à alteração.

Questão 2 — Qual problema ele procura resolver?
R: Ele procura resolver o problema de comunicação entre objetos quando vários componentes precisam ser avisados sobre mudanças. O padrão reduz o acoplamento porque o Subject não precisa conhecer os detalhes de cada reação.

Questão 3 — Qual é a responsabilidade da interface Observer?
R: Definir o contrato que todo observador deve cumprir. No laboratório, esse contrato é o método atualizar(status), responsável por receber a informação sobre o novo estado.

Questão 4 — Qual é a responsabilidade da interface Subject?
R: Definir as operações básicas do objeto observado: adicionar observadores, remover observadores e notificar os observadores cadastrados.

Questão 5 — Qual é a responsabilidade de Pedido?
R: Pedido é o Subject concreto. Ele controla seu status, mantém a lista de observadores, permite adicionar e remover observadores e dispara a notificação quando seu estado muda.

Questão 6 — Por que Pedido não conhece EmailObserver, SMSObserver ou LogObserver diretamente?
R: Porque Pedido depende da abstração Observer, e não das implementações concretas. Isso reduz o acoplamento e permite adicionar novos tipos de observadores sem alterar a classe Pedido.

Questão 7 — O que acontece quando adicionamos WhatsAppObserver?
R: Criamos uma nova classe que implementa Observer e a adicionamos ao Pedido. Quando o status mudar, WhatsAppObserver receberá atualizar(status). As classes existentes permanecem intactas.

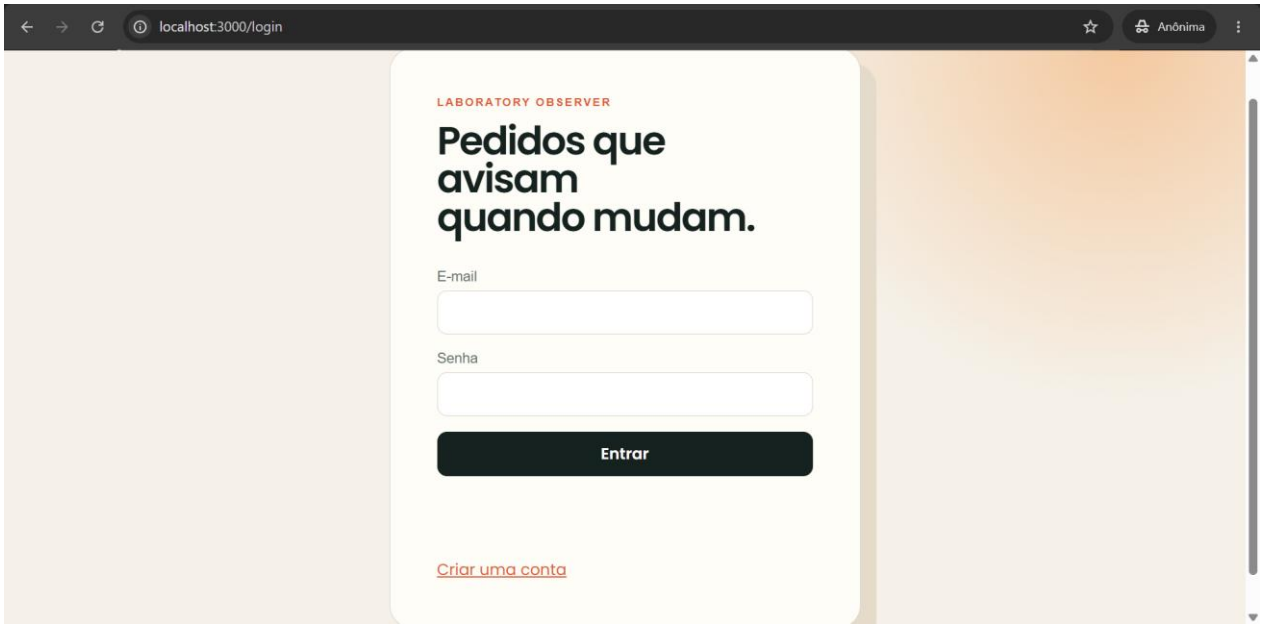
Questão 8 — Qual é a vantagem de poder adicionar e remover observadores em tempo de execução?
R: O sistema fica mais flexível porque a lista de interessados pode ser alterada dinamicamente. Podemos ativar ou desativar determinado mecanismo de notificação sem modificar a implementação do Subject.

Questão 9 — Cite um exemplo do mundo real, fora da programação, que se pareça com Observer.
R: Um canal de notícias é um bom exemplo. O canal representa o Subject e os assinantes representam os Observers. Quando uma nova notícia é publicada, todos os assinantes cadastrados recebem a atualização. Um assinante pode entrar ou sair da lista sem alterar a forma como o canal publica as notícias.

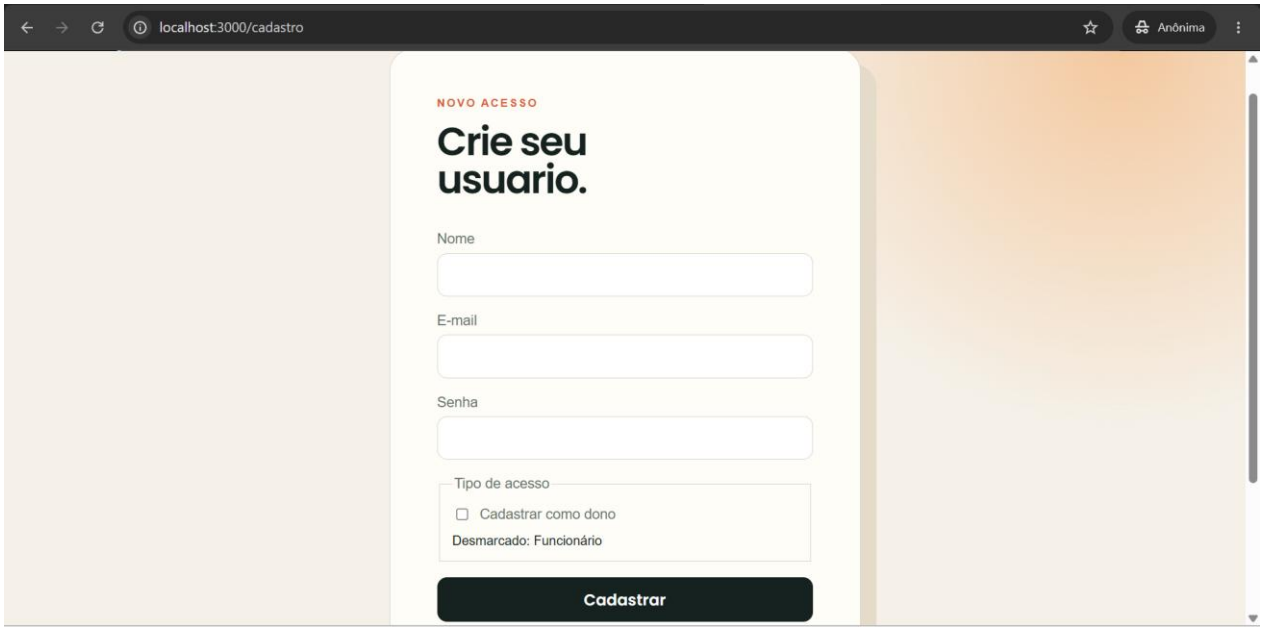
Questão 10 — Compare Observer com Strategy e Factory Method.
R: Strategy organiza alternativas de comportamento; Factory Method organiza alternativas de criação de objetos; Observer organiza a notificação de múltiplos dependentes quando ocorre uma mudança de estado. Assim, os três padrões utilizam abstrações para reduzir acoplamento, mas resolvem problemas diferentes.

Conclusão
R: O padrão Observer permite separar a alteração do estado de um objeto das ações que devem ocorrer em resposta a essa alteração. No laboratório, Pedido atua como Subject e os mecanismos de e-mail, SMS, log e WhatsApp atuam como Observers. A principal vantagem é que novos observadores podem ser adicionados sem modificar Pedido, favorecendo baixo acoplamento, flexibilidade, extensibilidade e o princípio Open/Closed Principle (OCP).

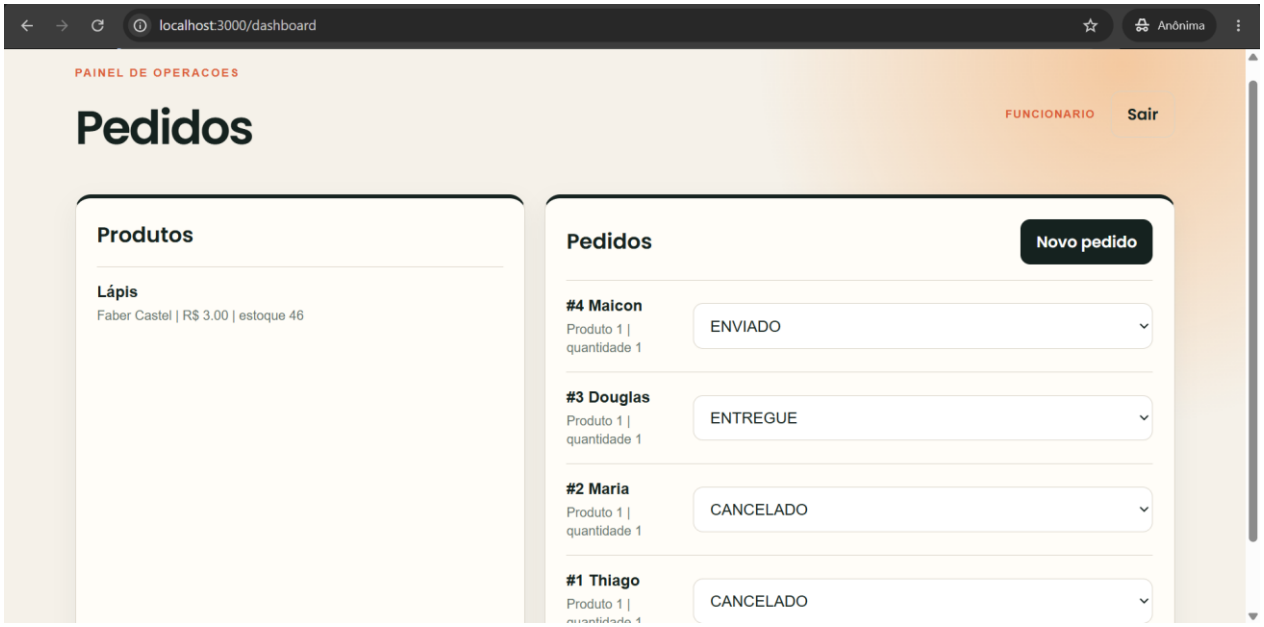
Print do funcionamento do projeto
Login



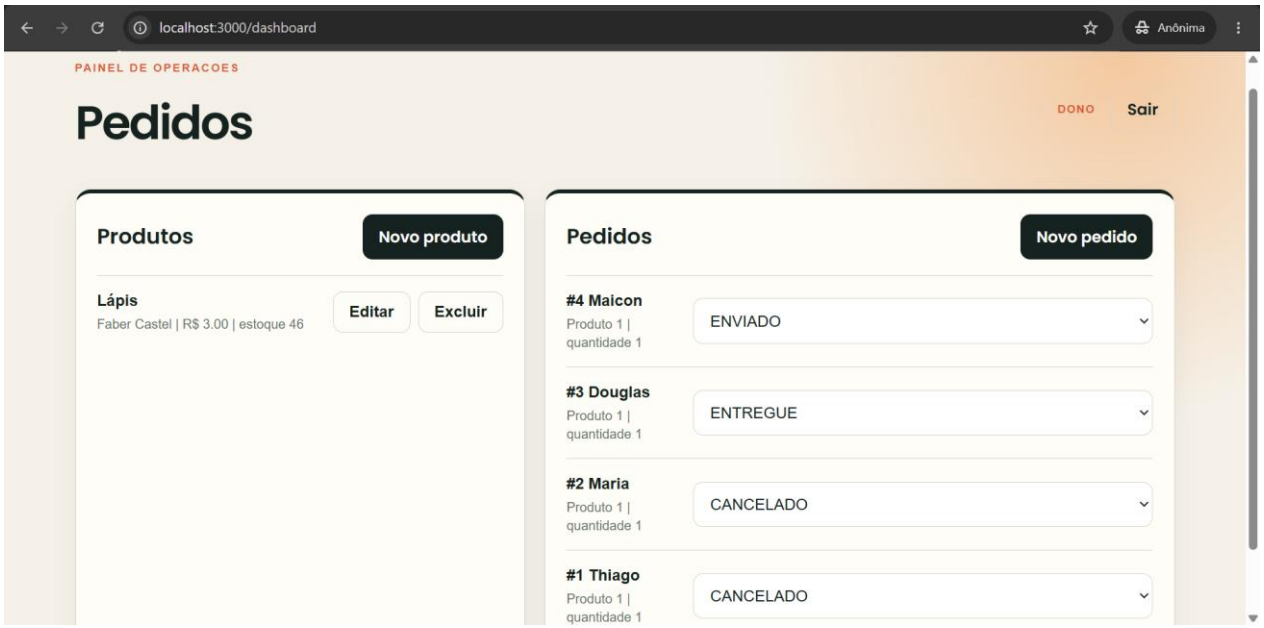
Cadastro



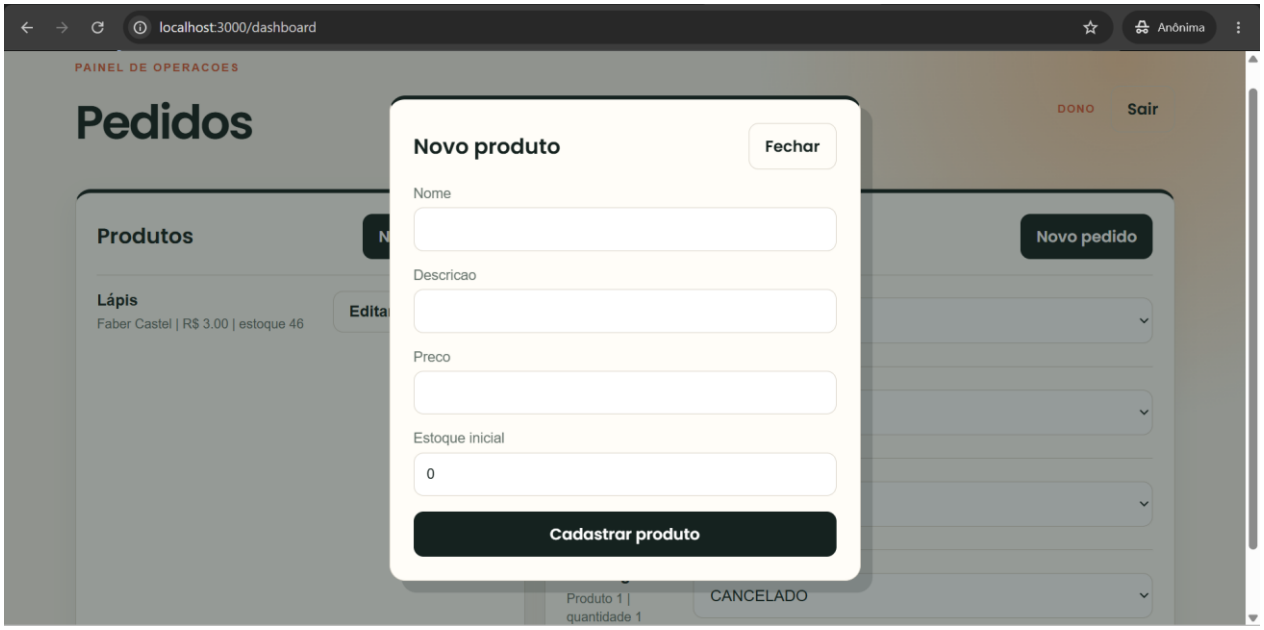
Tela do funcionário



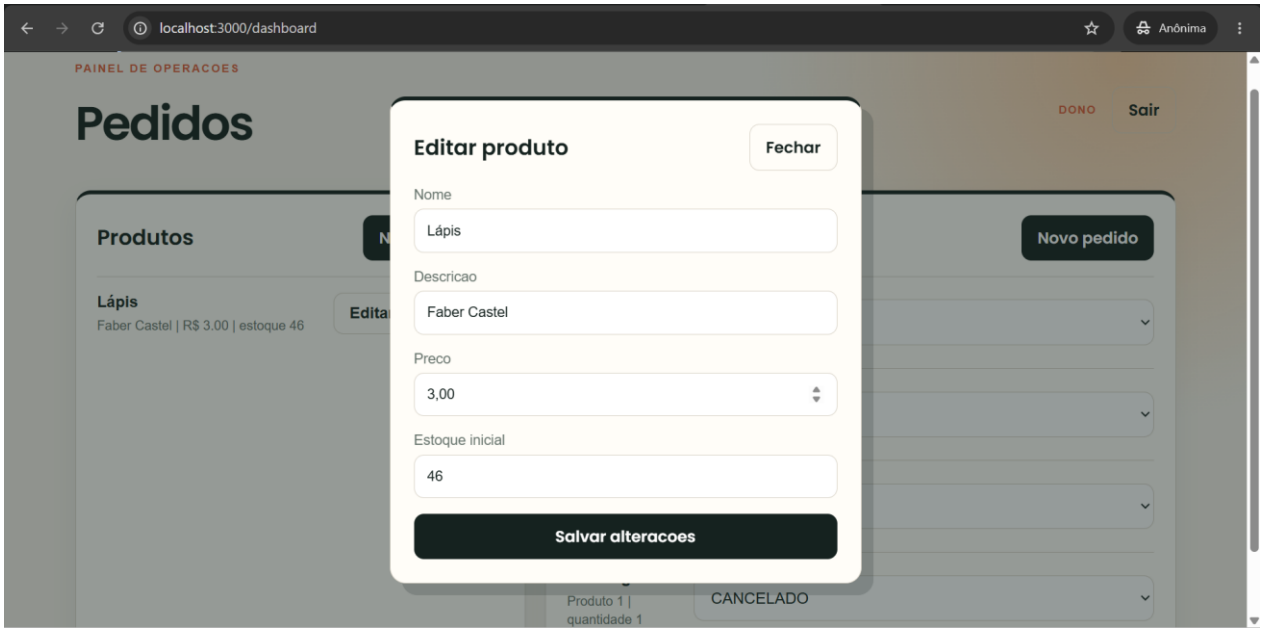
Tela do dono



Novo produto



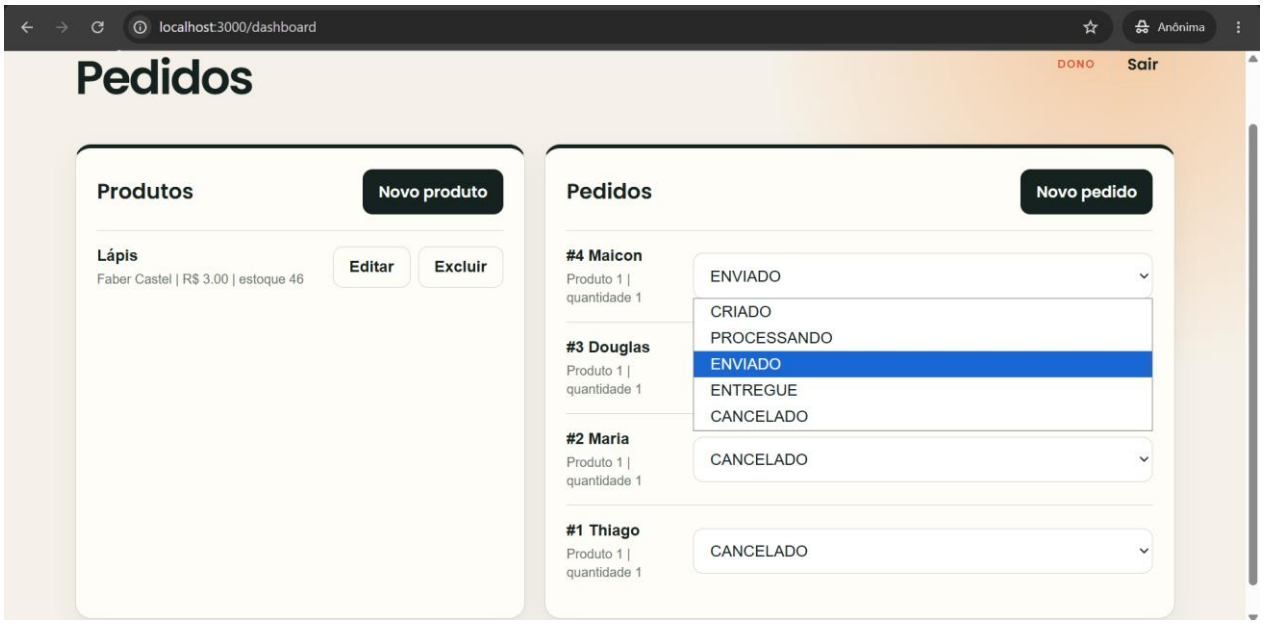
Editar produto



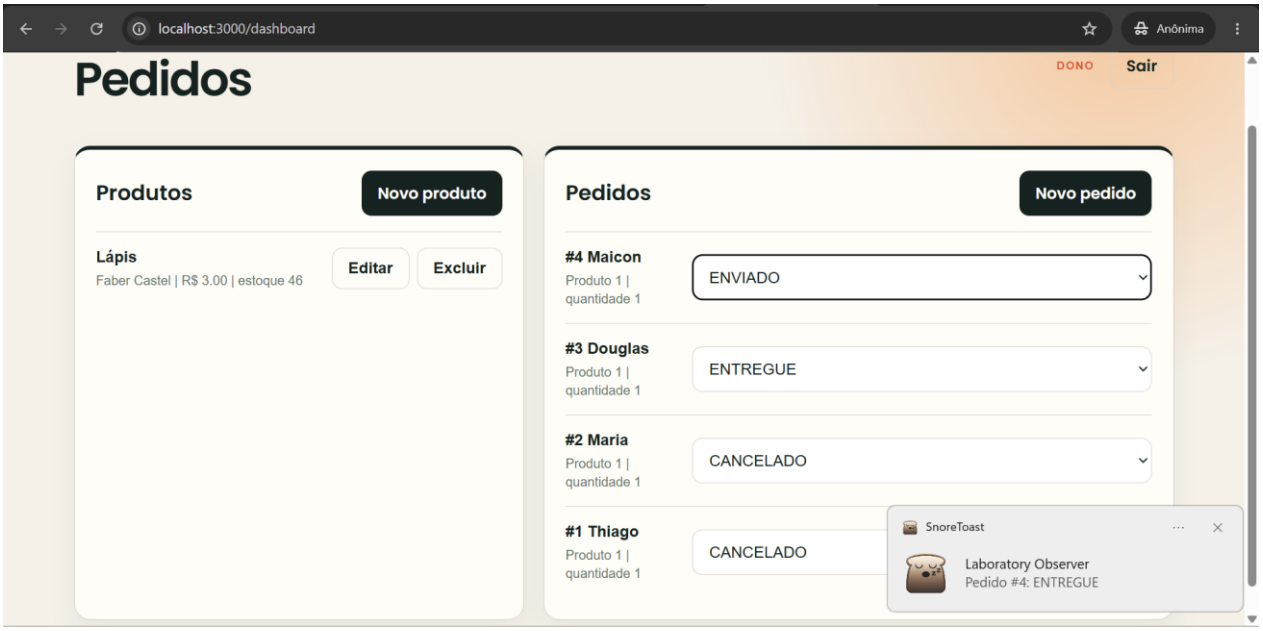
Novo pedido



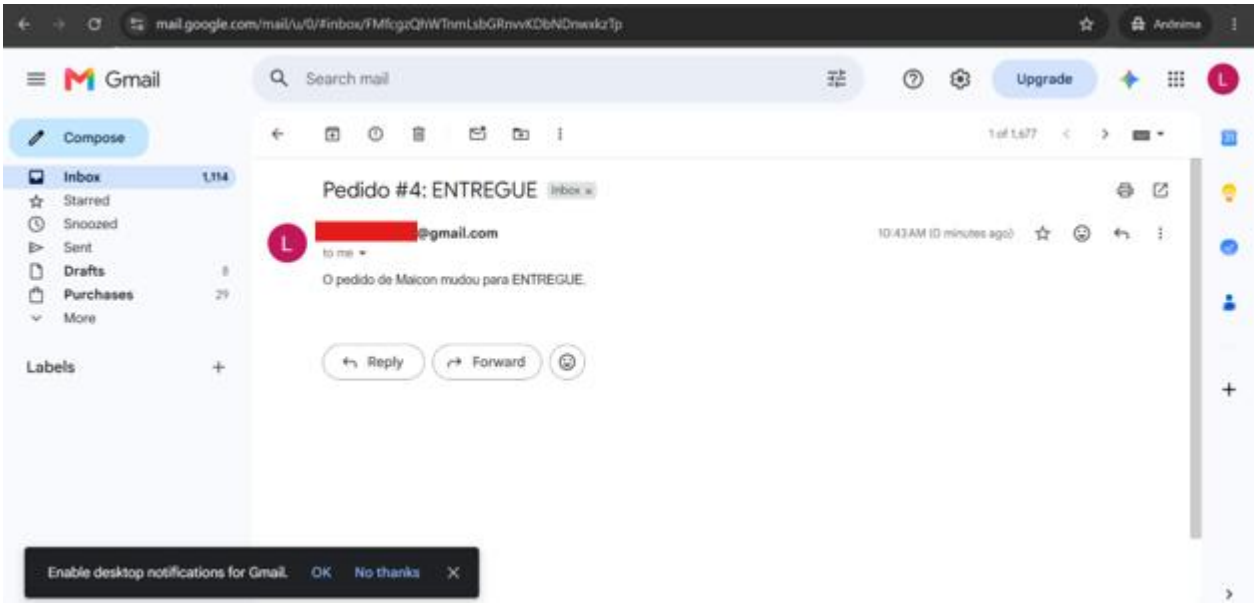
Fluxo do processo



Notificação do Sistema

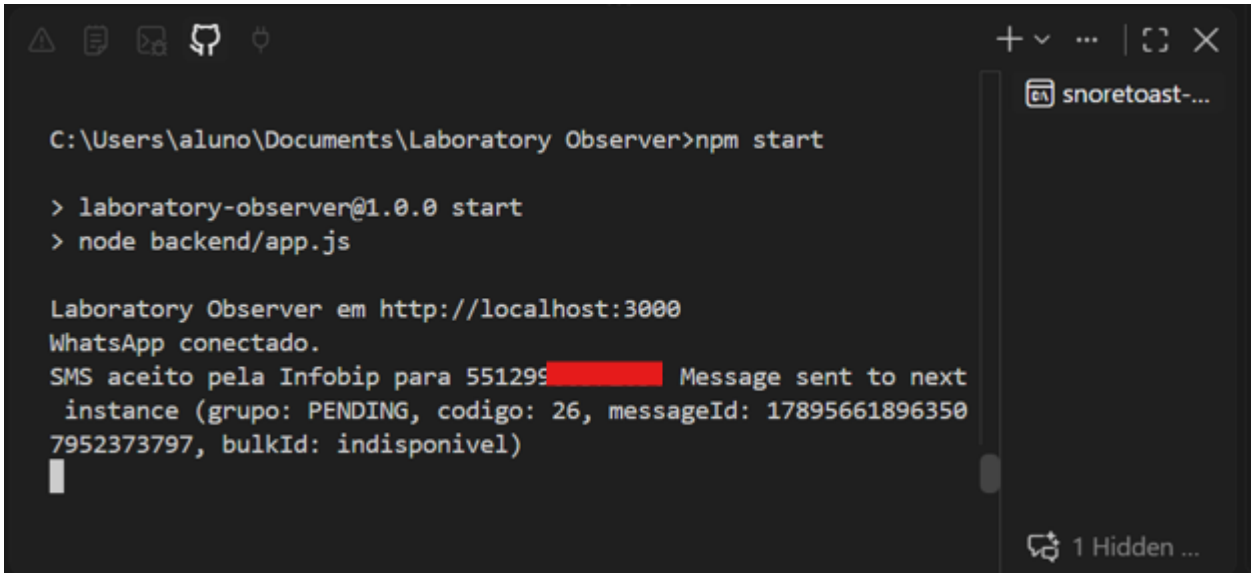


Notificação do e-mail



Notificação do SMS

Funciona, como pode ser visto no terminal. No entanto, a API da Infobip que estou utilizando acabou estourando a quantidade de usos.



Notificação do WhatsApp

